

PAYMENT TRACK, DAY 2: the course owner's share, BACKEND

E-Learning Platform (Spring Boot 3.2.5, Java 17). **8 new files, 1 file replaced, 1 line in `application.properties`**. Two new tables (`course_earnings` and `payouts` ; the Day 1 `payments` table is only read). No new Maven dependency. The SRS endpoints and models are untouched.

Prerequisites: Day 1 works (a test payment enrolls a customer), and the instructor module from the main plan's Day 2 is in (it provides the `course_owners` table that says who made each course).

By the end of Day 2:

- when a customer pays for a course made by an instructor, the instructor gets an **earning** (the price minus the platform's commission);
- the instructor can open "My earnings" and see every sale;
- the admin sees who is owed money and records a **payout** with the bank or UPI reference;
- a course made by the admin earns nothing for anyone (the platform keeps it).

0. The honest truth about "the payment goes to the course owner"

Read this first, because it decides what we build.

When a student pays in Razorpay, the money lands in **one** place: the Razorpay account of the **platform** (the account whose keys you put in `application.properties`). Razorpay does not know about your instructors. So "the payment goes to the owner" can mean two different things:

Meaning	What it needs	Built?
A. The platform keeps the money, records who earned what, and pays the instructor later (bank transfer or UPI), keeping a commission. This is how most small marketplaces start.	only your code	Yes, this guide
B. Razorpay splits the money automatically at payment time and sends the instructor's share straight to the instructor's own account (Razorpay Route).	Route enabled on your Razorpay account (Razorpay has to approve it), every instructor registered as a "linked account" with business details and KYC, and extra API calls	No. Not testable without those approvals. See section 7 for how to move there later

For your project demo, A shows the whole story: the student pays, the instructor sees the earning, the admin pays him out and records it. In test mode nothing is real anyway.

1. The ideas, from zero (tied to your code)

Ledger. A ledger is a notebook where every line is one fact that never changes: "on this payment, for this course, the student paid X, the platform keeps Y, the instructor is owed Z". Your `course_earnings` table is

that notebook. Balances are not stored; they are **added up** from the lines, so they can never drift from the truth.

Paise, not rupees. Computers add decimals badly ($0.1 + 0.2$ is not 0.3). So every amount is a whole number of **paise** (100 paise = Rs 1) in a `long`. Rs 499 is `49900`. The screen divides by 100. This is the same unit Razorpay uses (`payments.amount_paise`).

Commission. `commission.percent` (default 20) is the platform's share of each sale. `Money.fee(gross, percent)` takes that percent and rounds half up to a whole paise. The instructor gets the rest (`gross - fee`), so the two always add up to the gross.

Splitting one payment over several courses (`Money.allocate`). A cart can hold 3 courses and the student pays one total. We cut the paid total into one share per course, in proportion to the course prices. Dividing whole numbers leaves leftover paise (100 split three ways is $33+33+33 = 99$). The "largest remainder" method hands the left-over paise to the parts with the biggest fractions, so the parts **always add up to exactly the amount paid**. We split what was **actually paid** (`payments.amount_paise`), not today's prices, so if you edit a course price later, old earnings stay correct.

Whose course is it? `course_owners` (from the instructor module) has one row per instructor-made course. `EarningService.record` looks each course up there; no row means an admin course, and no earning is written.

Run twice, same result (idempotent). The browser may call `/verify` twice (double click). `record` first asks "do earnings for this payment already exist?" and also the table has a **unique key (payment, course)**, so even two requests at the same moment cannot write a line twice.

Payment first, bookkeeping second. `PaymentService.afterPaid` runs **after** the money is verified and the student is enrolled. If the ledger write fails, the student is **still enrolled** (the error is logged) and the next verify call writes it. A bookkeeping bug must never take away something a student paid for.

Payout. When the admin has paid an instructor outside the app, he types the transaction reference and presses "Mark as paid". `EarningService.pay` (in one database transaction) creates one `payouts` row and marks all of that instructor's pending lines as `PAID_OUT`. Paying twice is refused ("Nothing is pending").

2. What goes where

File	What it is
<code>pay/Money.java</code>	all the money arithmetic (no Spring, easy to test)
<code>model/CourseEarning.java</code>	one ledger line (new table <code>course_earnings</code>)
<code>model/Payout.java</code>	one payout (new table <code>payouts</code>)
<code>repository/CourseEarningRepo.java</code> , <code>PayoutRepo.java</code>	data access
<code>dto/PayDtos.java</code>	the shapes the screens receive (also holds the invoice shapes used on Day 3)
<code>service/EarningService.java</code>	record, summary, payable, pay

File	What it is
controller/EarningsController.java	GET /api/instructor/earnings, GET /api/admin/payouts/payable, GET /api/admin/payouts, POST /api/admin/payouts
service/PaymentService.java	replaced (adds the afterPaid call)

2a. application.properties : add at the very end

```
# Day 2: the platform's share of each sale, in percent (0 to 100). The instructor gets the rest.
commission.percent=${COMMISSION_PERCENT:20}
```

2b. SecurityConfig.java : nothing to do

/api/instructor/** is already INSTRUCTOR-only and /api/admin/** is ADMIN-only in the final SecurityConfig (master plan section 8), so the four new endpoints are protected without a new rule. An instructor cannot see another instructor's earnings because the service always uses the **logged-in** user, never an id from the request.

3. Complete code

springapp/src/main/java/com/examly/springapp/pay/Money.java (new file)

```
package com.examly.springapp.pay;

import java.util.ArrayList;
import java.util.List;

/**
 * All the money arithmetic of the payment features, in one small class with no Spring in it, so it is easy
 * to test.
 * Every amount is a whole number of paise (100 paise = 1 rupee). No decimals, so no rounding drift.
 */
public final class Money {
    private Money() {}

    /**
     * Splits {@code total} over the given weights in proportion, and guarantees the parts add up to
     * exactly
     * {@code total} (the "largest remainder" method). Example: 100 over [1,1,1] gives [34,33,33].
     */
    public static List<Long> allocate(long total, List<Long> weights) {
        int n = weights.size();
        List<Long> out = new ArrayList<>();
        long sum = 0;
        for (Long w : weights) sum += (w == null ? 0 : w);
        if (n == 0) return out;
        if (sum <= 0) {
            // nothing to be proportional to: give it all to the first part
            for (int i = 0; i < n; i++) out.add(i == 0 ? total : 0L);
            return out;
        }
        long[] part = new long[n];
        long[] rem = new long[n];
        long used = 0;
        for (int i = 0; i < n; i++) {
            long w = weights.get(i) == null ? 0 : weights.get(i);
            // total * w can be large, but paise totals stay far below Long.MAX / weights
            part[i] = Math.floorDiv(total * w, sum);
            rem[i] = Math.floorMod(total * w, sum);
        }
    }
}
```

```

        used += part[i];
    }
    long left = total - used;
    while (left > 0) {
        int best = -1;
        for (int i = 0; i < n; i++) {
            if (best == -1 || rem[i] > rem[best]) best = i;
        }
        part[best]++;
        rem[best] = -1;
        left--;
    }
    for (long p : part) out.add(p);
    return out;
}

/** The platform's share of {@code gross}: percent of it, rounded to the nearest paise (half up). */
public static long fee(long gross, int percent) {
    if (percent <= 0) return 0;
    if (percent >= 100) return gross;
    return (gross * percent + 50) / 100;
}

/** Tax part of a price that ALREADY includes GST: total * rate / (100 + rate), rounded half up. */
public static long taxInside(long totalInclusive, int gstPercent) {
    if (gstPercent <= 0) return 0;
    return (totalInclusive * gstPercent + (100 + gstPercent) / 2) / (100 + gstPercent);
}

/** "12,34,567.50" (Indian digit grouping) from paise. */
public static String rupees(long paise) {
    boolean neg = paise < 0;
    long abs = Math.abs(paise);
    String whole = Long.toString(abs / 100);
    String frac = String.format("%02d", abs % 100);
    String grouped;
    if (whole.length() <= 3) {
        grouped = whole;
    } else {
        String last3 = whole.substring(whole.length() - 3);
        String rest = whole.substring(0, whole.length() - 3);
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < rest.length(); i++) {
            if (i > 0 && (rest.length() - i) % 2 == 0) sb.append(',');
            sb.append(rest.charAt(i));
        }
        grouped = sb + "," + last3;
    }
    return (neg ? "-" : "") + grouped + "." + frac;
}
}

```

springapp/src/main/java/com/examly/springapp/model/CourseEarning.java (new file)

```

package com.examly.springapp.model;

import jakarta.persistence.*;
import java.time.Instant;

/**
 * One line of the instructor earnings ledger: "this paid order contained this instructor's course, so the
 * instructor is owed this much". New table "course_earnings"; nothing existing is changed.
 * Courses made by an admin have no owner, so they never get a row here (the platform keeps all of it).
 * Money is stored in paise (100 paise = 1 rupee) so there are no rounding surprises.
 */
@Entity
@Table(name = "course_earnings", uniqueConstraints = @UniqueConstraint(name = "uk_earning_payment_course",
    columnNames = {"paymentId", "courseId"}))
public class CourseEarning {

```

```

public static final String PENDING = "PENDING";
public static final String PAID_OUT = "PAID_OUT";

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false)
private Long paymentId;
private Long orderId;
@Column(nullable = false)
private Long courseId;
@Column(length = 200)
private String courseTitle;
@Column(nullable = false)
private Long instructorUserId;

/** What the customer paid for this course. */
private long grossPaise;
/** The platform's share. */
private long feePaise;
/** The instructor's share (gross minus fee). */
private long ownerPaise;

@Column(length = 20)
private String status;
private Long payoutId;
private Instant createdAt;

public CourseEarning() {}

public Long getId() { return id; }
public Long getPaymentId() { return paymentId; }
public void setPaymentId(Long v) { paymentId = v; }
public Long getOrderId() { return orderId; }
public void setOrderId(Long v) { orderId = v; }
public Long getCourseId() { return courseId; }
public void setCourseId(Long v) { courseId = v; }
public String getCourseTitle() { return courseTitle; }
public void setCourseTitle(String v) { courseTitle = v; }
public Long getInstructorUserId() { return instructorUserId; }
public void setInstructorUserId(Long v) { instructorUserId = v; }
public long getGrossPaise() { return grossPaise; }
public void setGrossPaise(long v) { grossPaise = v; }
public long getFeePaise() { return feePaise; }
public void setFeePaise(long v) { feePaise = v; }
public long getOwnerPaise() { return ownerPaise; }
public void setOwnerPaise(long v) { ownerPaise = v; }
public String getStatus() { return status; }
public void setStatus(String v) { status = v; }
public Long getPayoutId() { return payoutId; }
public void setPayoutId(Long v) { payoutId = v; }
public Instant getCreatedAt() { return createdAt; }
public void setCreatedAt(Instant v) { createdAt = v; }
}

```

springapp/src/main/java/com/examly/springapp/model/Payout.java (new file)

```

package com.examly.springapp.model;

import jakarta.persistence.*;
import java.time.Instant;

/**
 * A record that the admin paid an instructor everything that was pending at that moment.
 * The real money moves outside the app (bank transfer, UPI, RazorpayX); this row is the proof and the
 * reference.
 * New table "payouts".
 */

```

```

@Entity
@Table(name = "payouts")
public class Payout {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private Long instructorUserId;
    private long amountPaise;
    private int earningsCount;
    /** The bank / UPI transaction reference the admin typed in. */
    @Column(length = 100)
    private String reference;
    @Column(length = 120)
    private String paidByEmail;
    private Instant paidAt;

    public Payout() {}

    public Long getId() { return id; }
    public Long getInstructorUserId() { return instructorUserId; }
    public void setInstructorUserId(Long v) { instructorUserId = v; }
    public long getAmountPaise() { return amountPaise; }
    public void setAmountPaise(long v) { amountPaise = v; }
    public int getEarningsCount() { return earningsCount; }
    public void setEarningsCount(int v) { earningsCount = v; }
    public String getReference() { return reference; }
    public void setReference(String v) { reference = v; }
    public String getPaidByEmail() { return paidByEmail; }
    public void setPaidByEmail(String v) { paidByEmail = v; }
    public Instant getPaidAt() { return paidAt; }
    public void setPaidAt(Instant v) { paidAt = v; }
}

```

springapp/src/main/java/com/examly/springapp/repository/CourseEarningRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.CourseEarning;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for the instructor earnings ledger. */
@Repository
public interface CourseEarningRepo extends JpaRepository<CourseEarning, Long> {
    boolean existsByPaymentId(Long paymentId);
    List<CourseEarning> findByInstructorUserIdOrderByIdDesc(Long instructorUserId);
    List<CourseEarning> findByInstructorUserIdAndStatus(Long instructorUserId, String status);
    List<CourseEarning> findByStatus(String status);
}

```

springapp/src/main/java/com/examly/springapp/repository/PayoutRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.Payout;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for payouts. */
@Repository
public interface PayoutRepo extends JpaRepository<Payout, Long> {
    List<Payout> findByInstructorUserIdOrderByIdDesc(Long instructorUserId);
}

```

```
List<Payout> findAllByOrderByIdDesc();  
}
```

springapp/src/main/java/com/examly/springapp/dto/PayDtos.java (new file)

```
package com.examly.springapp.dto;  
  
import java.util.ArrayList;  
import java.util.List;  
  
/** Shapes of the earnings, payout and invoice APIs. Money is always a number of paise (100 paise = 1  
rupee). */  
public final class PayDtos {  
    private PayDtos() {}  
  
    public static class EarningRow {  
        public Long id;  
        public Long courseId;  
        public String courseTitle;  
        public Long paymentId;  
        public long grossPaise;  
        public long feePaise;  
        public long ownerPaise;  
        public String status;  
        public String createdAt;  
    }  
  
    public static class PayoutView {  
        public Long id;  
        public Long instructorUserId;  
        public String instructorName;  
        public long amountPaise;  
        public int earningsCount;  
        public String reference;  
        public String paidByEmail;  
        public String paidAt;  
    }  
  
    /** What an instructor sees on the earnings page. */  
    public static class EarningsSummary {  
        public long totalPaise;  
        public long pendingPaise;  
        public long paidOutPaise;  
        public int commissionPercent;  
        public List<EarningRow> earnings = new ArrayList<>();  
        public List<PayoutView> payouts = new ArrayList<>();  
    }  
  
    /** One line of the admin's "who is owed money" table. */  
    public static class PayableRow {  
        public Long instructorUserId;  
        public String name;  
        public String email;  
        public String mobile;  
        public long pendingPaise;  
        public int pendingCount;  
    }  
  
    public static class PayoutRequest {  
        public Long instructorUserId;  
        public String reference;  
    }  
  
    public static class InvoiceLineView {  
        public Long courseId;  
        public String title;  
        public long paise;  
    }  
}
```

```

    public static class InvoiceView {
        public Long id;
        public String invoiceNumber;
        public Long paymentId;
        public Long orderId;
        public String razorpayPaymentId;
        public String customerName;
        public String customerEmail;
        public String issuedAt;
        public List<InvoiceLineView> lines = new ArrayList<>();
        public long taxablePaise;
        public long taxPaise;
        public long totalPaise;
        public int gstPercent;
        public String sellerName;
        public String sellerAddress;
        public String sellerGstin;
    }
}

```

springapp/src/main/java/com/examly/springapp/service/EarningService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.PayDtos.*;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.CourseEarning;
import com.examly.springapp.model.CourseOwner;
import com.examly.springapp.model.Payment;
import com.examly.springapp.model.Payout;
import com.examly.springapp.model.User;
import com.examly.springapp.pay.Money;
import com.examly.springapp.repository.CourseEarningRepo;
import com.examly.springapp.repository.CourseOwnerRepo;
import com.examly.springapp.repository.CourseRepo;
import com.examly.springapp.repository.PayoutRepo;
import com.examly.springapp.repository.UserRepo;
import java.time.Instant;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * Who earned what from a paid order.
 * <p>
 * All customer money lands in the platform's one Razorpay account. This service keeps a LEDGER that says
how much of
 * each paid course belongs to the instructor who made it (the rest, the commission, stays with the
platform).
 * The admin then pays the instructors outside the app and records a payout here. Courses made by an admin
have
 * no owner, so the platform keeps all of that money and no ledger row is written.
 */
@Service
public class EarningService {
    private static final Logger log = LoggerFactory.getLogger(EarningService.class);

    private final CourseEarningRepo earnings;

```

```

private final PayoutRepo payouts;
private final CourseOwnerRepo owners;
private final CourseRepo courses;
private final UserRepo users;
private final int commissionPercent;

public EarningService(CourseEarningRepo earnings, PayoutRepo payouts, CourseOwnerRepo owners,
CourseRepo courses,
                    UserRepo users, @Value("${commission.percent:20}") int commissionPercent) {
    this.earnings = earnings;
    this.payouts = payouts;
    this.owners = owners;
    this.courses = courses;
    this.users = users;
    this.commissionPercent = Math.max(0, Math.min(100, commissionPercent));
}

// ----- writing the ledger

/** Called once a payment is verified. Safe to call again for the same payment: it writes nothing the
second time. */
public int record(Payment payment, Long orderId) {
    if (payment == null || payment.getPaymentId() == null || payment.getCourseIds() == null) return 0;
    if (DatabaseOperationException.guard("checking earnings", () ->
earnings.existsByPaymentId(payment.getPaymentId())) {
        return 0;
    }
    List<Long> ids = new ArrayList<>();
    for (String s : payment.getCourseIds().split(",")) {
        if (!s.isBlank()) ids.add(Long.valueOf(s.trim()));
    }
    List<Course> list = new ArrayList<>();
    List<Long> weights = new ArrayList<>();
    for (Long id : ids) {
        Course c = DatabaseOperationException.guard("loading a course", () ->
courses.findById(id)).orElse(null);
        list.add(c);
        weights.add(c == null || c.getCoursePrice() == null ? 0L : Math.round(c.getCoursePrice() *
100));
    }
    // split what the customer really paid over the courses, so the lines add up to the payment exactly
    List<Long> gross = Money.allocate(payment.getAmountPaise(), weights);
    int written = 0;
    for (int i = 0; i < ids.size(); i++) {
        Long courseId = ids.get(i);
        Optional<CourseOwner> owner = DatabaseOperationException.guard("loading a course owner", () ->
owners.findById(courseId));
        if (owner.isEmpty()) continue; // admin course: the platform keeps
everything
        long g = gross.get(i);
        long fee = Money.fee(g, commissionPercent);
        CourseEarning e = new CourseEarning();
        e.setPaymentId(payment.getPaymentId());
        e.setOrderId(orderId);
        e.setCourseId(courseId);
        e.setCourseTitle(list.get(i) == null ? "Course " + courseId : list.get(i).getCourseType());
        e.setInstructorUserId(owner.get().getInstructorUserId());
        e.setGrossPaise(g);
        e.setFeePaise(fee);
        e.setOwnerPaise(g - fee);
        e.setStatus(CourseEarning.PENDING);
        e.setCreatedAt(Instant.now());
        try {
            earnings.save(e);
            written++;
        } catch (DataIntegrityViolationException dup) {
            log.info("Earning for payment {} course {} already exists", payment.getPaymentId(),
courseId);
        }
    }
    log.info("Payment {}: {} earning row(s) written", payment.getPaymentId(), written);
}

```

```

        return written;
    }

    // ----- instructor view

    public EarningsSummary forInstructor(User instructor) {
        List<CourseEarning> rows = DatabaseOperationException.guard("loading earnings",
            () -> earnings.findByInstructorUserIdOrderByDesc(instructor.getUserId()));
        EarningsSummary s = new EarningsSummary();
        s.commissionPercent = commissionPercent;
        for (CourseEarning e : rows) {
            s.totalPaise += e.getOwnerPaise();
            if (CourseEarning.PAID_OUT.equals(e.getStatus())) s.paidOutPaise += e.getOwnerPaise();
            else s.pendingPaise += e.getOwnerPaise();
            s.earnings.add(row(e));
        }
        for (Payout p : DatabaseOperationException.guard("loading payouts", () ->
            payouts.findByInstructorUserIdOrderByDesc(instructor.getUserId()))) {
            s.payouts.add(view(p, instructor.getUsername()));
        }
        return s;
    }

    // ----- admin view and payout

    /** Instructors who are owed money, with the amount. */
    public List<PayableRow> payable() {
        Map<Long, PayableRow> byInstructor = new LinkedHashMap<>();
        for (CourseEarning e : DatabaseOperationException.guard("loading earnings", () ->
            earnings.findByStatus(CourseEarning.PENDING))) {
            PayableRow r = byInstructor.computeIfAbsent(e.getInstructorUserId(), id -> {
                PayableRow n = new PayableRow();
                n.instructorUserId = id;
                User u = users.findById(id).orElse(null);
                n.name = u == null ? "User " + id : u.getUsername();
                n.email = u == null ? "" : u.getEmail();
                n.mobile = u == null ? "" : u.getMobileNumber();
                return n;
            });
            r.pendingPaise += e.getOwnerPaise();
            r.pendingCount++;
        }
        return new ArrayList<>(byInstructor.values());
    }

    public List<PayoutView> allPayouts() {
        List<PayoutView> out = new ArrayList<>();
        for (Payout p : DatabaseOperationException.guard("loading payouts", () ->
            payouts.findAllByOrderByDesc())) {
            User u = users.findById(p.getInstructorUserId()).orElse(null);
            out.add(view(p, u == null ? "User " + p.getInstructorUserId() : u.getUsername()));
        }
        return out;
    }

    /** Marks everything pending for this instructor as paid out, and keeps the bank / UPI reference the
    admin typed. */
    @Transactional
    public PayoutView pay(String adminEmail, PayoutRequest req) {
        if (req == null || req.instructorUserId == null) throw new InvalidRequestException("Choose an
        instructor");
        String ref = req.reference == null ? "" : req.reference.trim();
        if (ref.length() < 3 || ref.length() > 100) {
            throw new InvalidRequestException("Enter the bank or UPI transaction reference (3 to 100
            characters)");
        }
        User instructor = users.findById(req.instructorUserId)
            .orElseThrow(() -> new ResourceNotFoundException("Instructor not found"));
        if (!"INSTRUCTOR".equals(instructor.getRole())) throw new ResourceNotFoundException("Instructor not
        found");
        List<CourseEarning> pending = DatabaseOperationException.guard("loading earnings",

```

```

        () -> earnings.findByInstructorUserIdAndStatus(instructor.getUserId(),
CourseEarning.PENDING));
    if (pending.isEmpty()) throw new InvalidRequestException("Nothing is pending for this instructor");
    long sum = 0;
    for (CourseEarning e : pending) sum += e.getOwnerPaise();
    Payout p = new Payout();
    p.setInstructorUserId(instructor.getUserId());
    p.setAmountPaise(sum);
    p.setEarningsCount(pending.size());
    p.setReference(ref);
    p.setPaidByEmail(adminEmail);
    p.setPaidAt(Instant.now());
    Payout saved = DatabaseOperationException.guard("saving a payout", () -> payouts.save(p));
    for (CourseEarning e : pending) {
        e.setStatus(CourseEarning.PAID_OUT);
        e.setPayoutId(saved.getId());
    }
    DatabaseOperationException.guard("saving earnings", () -> earnings.saveAll(pending));
    log.info("Payout {}: {} paise to instructor {} ({} earnings) by {}", saved.getId(), sum,
instructor.getUserId(), pending.size(), adminEmail);
    return view(saved, instructor.getUsername());
}

// ----- mapping

private static EarningRow row(CourseEarning e) {
    EarningRow r = new EarningRow();
    r.id = e.getId();
    r.courseId = e.getCourseId();
    r.courseTitle = e.getCourseTitle();
    r.paymentId = e.getPaymentId();
    r.grossPaise = e.getGrossPaise();
    r.feePaise = e.getFeePaise();
    r.ownerPaise = e.getOwnerPaise();
    r.status = e.getStatus();
    r.createdAt = e.getCreatedAt() == null ? null : e.getCreatedAt().toString();
    return r;
}

private static PayoutView view(Payout p, String name) {
    PayoutView v = new PayoutView();
    v.id = p.getId();
    v.instructorUserId = p.getInstructorUserId();
    v.instructorName = name;
    v.amountPaise = p.getAmountPaise();
    v.earningsCount = p.getEarningsCount();
    v.reference = p.getReference();
    v.paidByEmail = p.getPaidByEmail();
    v.paidAt = p.getPaidAt() == null ? null : p.getPaidAt().toString();
    return v;
}
}

```

springapp/src/main/java/com/examly/springapp/controller/EarningsController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.PayDtos.*;
import com.examly.springapp.service.AccessService;
import com.examly.springapp.service.EarningService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;

```

```

import org.springframework.web.bind.annotation.RestController;

/**
 * Instructor earnings (INSTRUCTOR only) and payouts (ADMIN only). The role rules live in SecurityConfig:
 * /api/instructor/** is INSTRUCTOR and /api/admin/** is ADMIN, so no new security rule is needed for
 * these.
 */
@RestController
@RequestMapping("/api")
@Tag(name = "Earnings", description = "Instructor earnings and admin payouts")
public class EarningsController {
    private final EarningService service;
    private final AccessService access;

    public EarningsController(EarningService service, AccessService access) {
        this.service = service;
        this.access = access;
    }

    @Operation(summary = "My earnings: totals, every sale, and the payouts I received")
    @GetMapping("/instructor/earnings")
    public ResponseEntity<EarningsSummary> mine() {
        return ResponseEntity.ok(service.forInstructor(access.currentUser()));
    }

    @Operation(summary = "Instructors who are owed money")
    @GetMapping("/admin/payouts/payable")
    public ResponseEntity<List<PayableRow>> payable() {
        return ResponseEntity.ok(service.payable());
    }

    @Operation(summary = "All payouts made so far")
    @GetMapping("/admin/payouts")
    public ResponseEntity<List<PayoutView>> history() {
        return ResponseEntity.ok(service.allPayouts());
    }

    @Operation(summary = "Record that an instructor was paid everything pending")
    @PostMapping("/admin/payouts")
    public ResponseEntity<PayoutView> pay(@RequestBody PayoutRequest body) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.pay(access.currentEmail(), body));
    }
}

```

springapp/src/main/java/com/examly/springapp/service/PaymentService.java (REPLACE the whole Day 1 file with this one. Only 4 things changed: the constructor takes an `EarningService`, the `afterPaid` method is new, and it is called twice)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.RazorpayVerifyDTO;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.DuplicateResourceException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Cart;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.Customer;
import com.examly.springapp.model.Orders;
import com.examly.springapp.model.Payment;
import com.examly.springapp.repository.OrderRepo;
import com.examly.springapp.repository.PaymentRepo;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashMap;

```

```

import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;

/**
 * Pay for the cart with Razorpay (test mode or live mode, decided only by the keys you configure).
 * <p>
 * Step 1, {@link #createCheckout}: the SERVER reads the customer's cart, adds up the prices from the
database
 * (the browser can never choose the price), creates an order at Razorpay and remembers it in the payments
table.
 * <br>
 * Step 2, {@link #verifyAndEnroll}: after the customer pays, the server checks Razorpay's signature. Only
then it saves
 * the real order through the existing OrderService, so the duplicate-purchase rule still applies, and
empties the cart.
 */
@Service
public class PaymentService {
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);
    /** Razorpay cannot charge less than 1 rupee (100 paise). */
    private static final long MIN_PAISE = 100;

    private final RazorpayClient razorpay;
    private final CartService cartService;
    private final OrderService orderService;
    private final OrderRepo orderRepo;
    private final PaymentRepo paymentRepo;
    private final EarningService earningService;

    public PaymentService(RazorpayClient razorpay, CartService cartService, OrderService orderService,
        OrderRepo orderRepo, PaymentRepo paymentRepo,
        EarningService earningService) {
        this.razorpay = razorpay;
        this.cartService = cartService;
        this.orderService = orderService;
        this.orderRepo = orderRepo;
        this.paymentRepo = paymentRepo;
        this.earningService = earningService;
    }

    /** Creates the Razorpay order for what is in the customer's cart and returns what the browser needs to
open Checkout. */
    public Map<String, Object> createCheckout(Customer customer) {
        log.trace("createCheckout entered");
        requireConfigured();
        Long customerId = customer.getCustomerId();
        Cart cart = cartService.getCartByCustomerId(customerId);
        List<Course> courses = cart.getCourses();
        if (courses == null || courses.isEmpty()) {
            throw new InvalidRequestException("Your cart is empty");
        }
        // same rule as the order service: a course can be bought only once
        Set<Long> bought = new HashSet<>();
        DatabaseOperationException.guard("loading orders", () ->
orderRepo.findByCustomerId(customerId))
            .forEach(o -> o.getCourses().forEach(c -> bought.add(c.getCourseId())));
        BigDecimal total = BigDecimal.ZERO;
        for (Course c : courses) {
            if (bought.contains(c.getCourseId())) {
                throw new DuplicateResourceException("You are already enrolled in " + c.getCourseType());
            }
            total = total.add(BigDecimal.valueOf(c.getCoursePrice() == null ? 0 : c.getCoursePrice()));
        }
        long paise = total.movePointRight(2).setScale(0, RoundingMode.HALF_UP).longValueExact();
        if (paise < MIN_PAISE) {

```

```

        throw new InvalidRequestException("Razorpay needs a total of at least Rs 1. A free cart does
not need a payment.");
    }

    String receipt = ("rcpt_" + customerId + "_" + System.currentTimeMillis());
    String razorpayOrderId = razorpay.createOrder(paise, receipt.length() > 40 ? receipt.substring(0,
40) : receipt);

    Payment payment = new Payment();
    payment.setRazorpayOrderId(razorpayOrderId);
    payment.setAmountPaise(paise);
    payment.setCurrency("INR");
    payment.setStatus(Payment.CREATED);
    payment.setCustomerId(customerId);
    payment.setCourseIds(courses.stream().map(c ->
String.valueOf(c.getCourseId())).collect(Collectors.joining(",")));
    payment.setCreatedAt(Instant.now());
    DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));

    Map<String, Object> prefill = new LinkedHashMap<>();
    prefill.put("name", customer.getCustomerName());
    if (customer.getUser() != null) {
        prefill.put("email", customer.getUser().getEmail());
        String mobile = customer.getUser().getMobileNumber();
        if (mobile != null && mobile.matches("^([6-9][0-9]{9}$)")) {
            prefill.put("contact", mobile);
        }
    }
    Map<String, Object> out = new LinkedHashMap<>();
    out.put("keyId", razorpay.getKeyId());
    out.put("razorpayOrderId", razorpayOrderId);
    out.put("amount", paise);
    out.put("currency", "INR");
    out.put("name", "Online E-Learning Platform");
    out.put("description", courses.size() + (courses.size() == 1 ? " course" : " courses"));
    out.put("prefill", prefill);
    log.info("Checkout prepared: customer {}, {} course(s), {} paise", customerId, courses.size(),
paise);
    return out;
}

/** Checks the signature, then saves the real order. Calling it twice with the same payment returns the
same order. */
public Orders verifyAndEnroll(Customer customer, RazorpayVerifyDTO dto) {
    log.trace("verifyAndEnroll entered");
    requireConfigured();
    Payment payment = DatabaseOperationException.guard("loading a payment", () ->
paymentRepo.findByRazorpayOrderId(dto.razorpayOrderId()))
        .orElseThrow(() -> new ResourceNotFoundException("Payment not found"));
    if (!payment.getCustomerId().equals(customer.getCustomerId())) {
        log.warn("Payment {} belongs to another customer", payment.getPaymentId());
        throw new AccessDeniedException("This payment belongs to another customer");
    }
    if (Payment.PAID.equals(payment.getStatus()) && payment.getOrderId() != null) {
        // the browser repeated the call (double click, reload): answer with the order we already saved
        log.info("Payment {} was already verified; returning order {}", payment.getPaymentId(),
payment.getOrderId());
        afterPaid(payment, payment.getOrderId()); // heals an earning that failed the first time
        return orderService.getOrderBy(payment.getOrderId());
    }
    if (!RazorpaySignature.matches(dto.razorpayOrderId(), dto.razorpayPaymentId(),
dto.razorpaySignature(), razorpay.getKeySecret())) {
        log.warn("Payment {}: signature check FAILED", payment.getPaymentId());
        throw new InvalidRequestException("Payment verification failed. You were not enrolled.");
    }

    // the signature is valid: the money is real. Record it before anything else can go wrong.
    payment.setRazorpayPaymentId(dto.razorpayPaymentId());
    payment.setStatus(Payment.PAID);
    payment.setPaidAt(Instant.now());
    DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));
}

```

```

Orders order = new Orders();
order.setCustomer(customer);
List<Course> stubs = new ArrayList<>();
for (String id : payment.getCourseIds().split(",")) {
    Course c = new Course();
    c.setCourseId(Long.valueOf(id.trim()));
    stubs.add(c);
}
order.setCourses(stubs);
Orders saved;
try {
    saved = orderService.addOrder(order);    // recalculates the price from the database and applies
the duplicate rule
} catch (RuntimeException e) {
    payment.setStatus(Payment.PAID_ORDER_FAILED);
    paymentRepo.save(payment);
    log.error("Payment {} (Razorpay payment id {}) was paid but the enrollment failed: {}",
        payment.getPaymentId(), dto.razorpayPaymentId(), e.getMessage());
    throw e;
}
payment.setOrderId(saved.getOrderId());
DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));

try {

cartService.removeAllCourses(cartService.getCartByCustomerId(customer.getCustomerId()).getCartId());
    } catch (RuntimeException e) {
        log.warn("Order {} saved but the cart could not be emptied: {}", saved.getOrderId(),
e.getMessage());
    }
    afterPaid(payment, saved.getOrderId());
    log.info("Payment {} verified: order {} created for customer {}", payment.getPaymentId(),
saved.getOrderId(), customer.getCustomerId());
    return saved;
}

/**
 * The instructor earnings come AFTER the money and the enrollment are safe. If it fails, the
 * customer is still enrolled; the error is logged and the next verify call writes the earnings again.
 */
private void afterPaid(Payment payment, Long orderId) {
    try {
        earningService.record(payment, orderId);
    } catch (RuntimeException e) {
        log.error("Payment {}: the instructor earnings could not be recorded: {}",
payment.getPaymentId(), e.getMessage());
    }
}

private void requireConfigured() {
    if (!razorpay.isConfigured()) {
        throw new InvalidRequestException("Razorpay is not set up on this server");
    }
}
}

```

4. Check it

4a. Start the backend. There must be no red error. Hibernate creates `course_earnings` and `payouts`.

4b. One full sale (Swagger or the Angular app). 1. Log in as an **approved instructor**, create a course "Java Basics" at price 500 (the instructor module does this; note it belongs to him). 2. Log in as a customer, add that course **and an admin-made course** to the cart, and pay with a test card (Day 1). 3. Check the database:

```
SELECT id, payment_id, course_id, course_title, instructor_user_id, gross_paise, fee_paise, owner_paise,
status
FROM course_earnings ORDER BY id DESC;
SELECT * FROM payouts;
```

You should see **one** row, for the instructor's course only: `gross_paise 50000`, `fee_paise 10000`, `owner_paise 40000`, `status PENDING`. The admin course produced **no** row.

4c. Things to try (all should behave as described; these are what the tests checked).

Try	Expected
Pay a cart of 2 instructor courses and 1 admin course	2 earning rows, the 3rd course earns nothing
Call <code>POST /api/payment/razorpay/verify</code> twice with the same payment	still the same number of earning rows
Change a course price after paying	old earning rows keep the amounts that were paid
<code>GET /api/instructor/earnings</code> as that instructor	<code>totalPaise</code> , <code>pendingPaise</code> , <code>paidOutPaise</code> , <code>commissionPercent</code> , <code>earnings[]</code> , <code>payouts[]</code>
<code>GET /api/instructor/earnings</code> as a customer or admin	403
<code>GET /api/admin/payouts/payable</code> as admin	one row per instructor who is owed money, with the amount
<code>POST /api/admin/payouts</code> with <code>{"instructorUserId": 12, "reference": "UPI-9981"}</code>	201, the amount equals the pending total; his lines become <code>PAID_OUT</code>
The same request again	400 "Nothing is pending for this instructor"
reference <code>"ab"</code>	400 "Enter the bank or UPI transaction reference (3 to 100 characters)"
<code>instructorUserId</code> of a customer, or of nobody	404 "Instructor not found"

5. If something goes wrong

What you see	Why	Fix
Start-up error "No qualifying bean of type EarningService" or "...CourseOwnerRepo"	a file is missing, or the instructor module (Day 2 of the main plan) is not in	add the missing file; <code>CourseOwnerRepo</code> comes from the instructor module
Payment works but <code>course_earnings</code> stays empty	the course has no row in <code>course_owners</code> (it is an admin course), or the log shows "instructor earnings could not be recorded"	check <code>SELECT * FROM course_owners</code> ; ; read the log line
Pay again with the same payment id and nothing new appears	correct: it is safe to repeat	none
403 on <code>/api/admin/payouts</code>	you are not logged in as ADMIN	use the admin token
Table 'course_earnings' doesn't exist	<code>ddl-auto</code> is not <code>update</code>	set <code>DDL_AUTO=update</code>

6. Security rules for the team

- The server decides the owner share. The browser never sends an amount or an instructor id for earnings.
- An instructor sees only his own earnings (the service uses the logged-in user).
- Only an ADMIN can record a payout. A payout is recorded **after** the real transfer, with its bank or UPI reference, so the books match the bank.
- Ledger lines are never edited or deleted; only their status changes from `PENDING` to `PAID_OUT`.

7. Moving to automatic split later (Razorpay Route), and what is not built

- **Route (meaning B above).** The change would be: register each instructor as a linked account at Razorpay (needs his business or personal KYC details), store the linked account id, and add a `transfers` list (account id, amount) when the order is created. The amounts to send are exactly what `EarningService.record` already computes, so this ledger stays useful as your audit trail. I did **not** build or test this because it needs approvals on your Razorpay account; check Razorpay's current Route documentation and whether it is available to you before you plan it.
- **Refunds:** if you refund a student, the instructor's earning must be reversed. Not built. Today a refund would have to be handled by hand.
- **Deleting a course** does not delete its earnings (they are history), but the course title is stored on each line, so the report still reads well.
- **Instructor bank or UPI details** are not stored in the app; the admin uses the e-mail and mobile number shown on the Payouts page and pays by his own bank app.
- **TDS / tax on payouts** to individuals: not handled. Ask your company's finance person; this is a legal matter, not code.

Honest status. The money arithmetic, the ledger, the instructor summary, the payout and the full pay-and-verify flow (including "the invoice or ledger fails but the student is still enrolled") ran as real Java against in-memory stand-ins: **136 checks, 0 failures** (this also covers Day 3). **Not tested:** Spring wiring (controller, tables, Security rules) against a real MySQL, and a real Razorpay payment.